

Practice 3

Network topology simulation in ns-3

Sangheon Pack
(shpack@korea.ac.kr)
2023.11.30

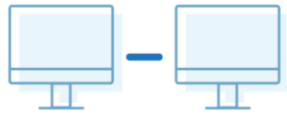
Network Topology

Network topology

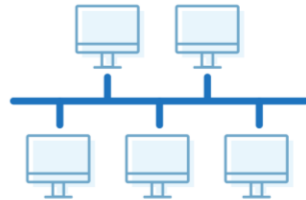
- **Network topology** is **physical layout** and **arrangement** of devices in networks
- It plays a crucial role in determining how data is **transmitted** and **shared** within the network
- When a **bottleneck** occurs, the **network's topology** can become a key factor in identifying issues.

Network topology types

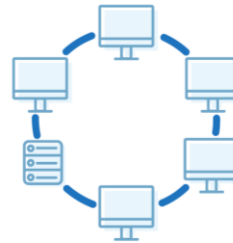
1 Point to point



2 Bus



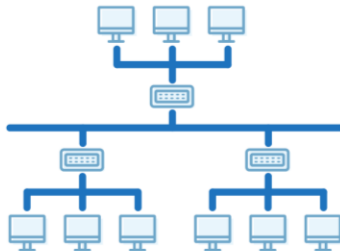
3 Ring



4 Star



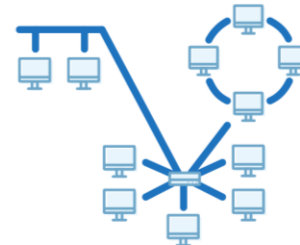
5 Tree



6 Mesh

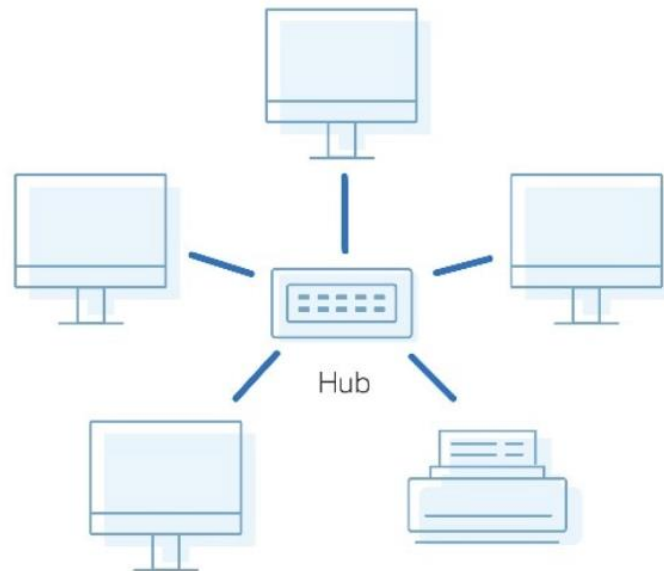


7 Hybrid



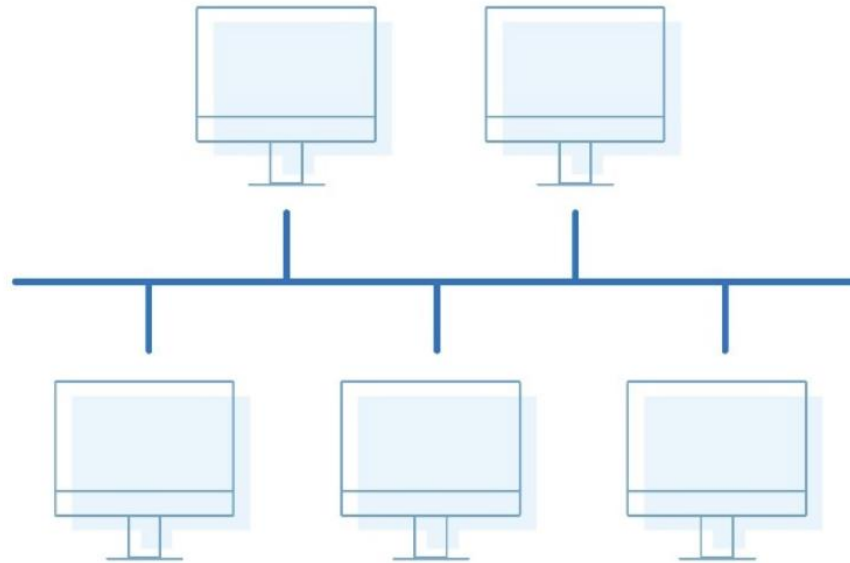
Star topology

- Network configuration where all devices are connected to a **central point**, typically represented by a hub or a switch



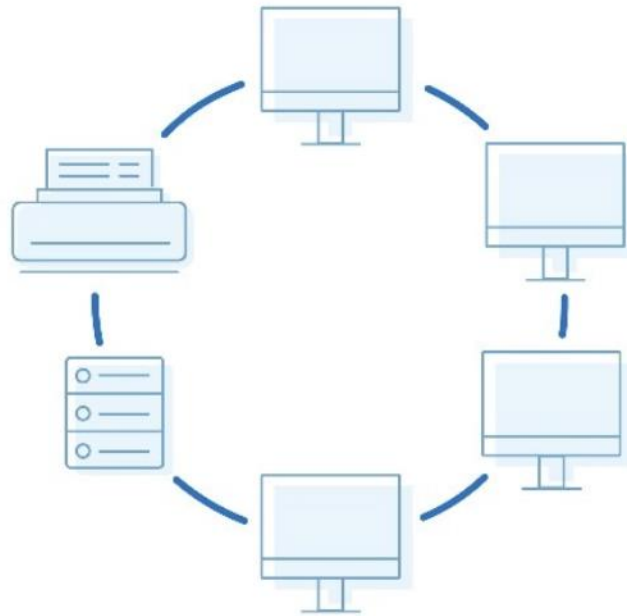
Bus topology

- Network configuration where all devices are connected in a **linear fashion**



Ring topology

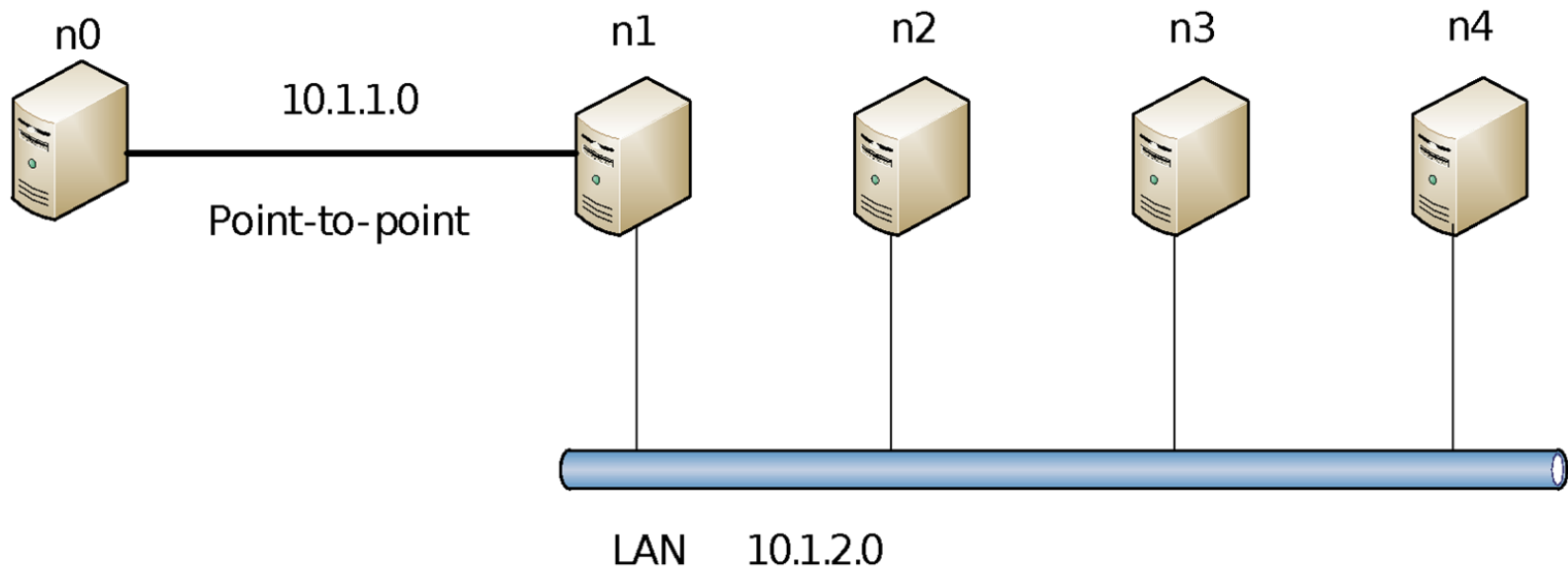
- Network configuration in which all devices are connected in a **circular loop**



Practice 2 Review

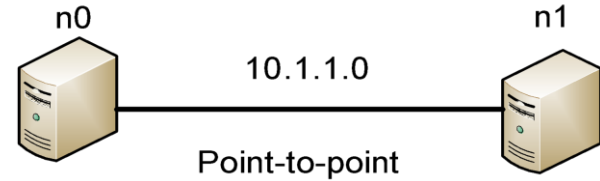
Review: CSMA topology

- CSMA network topology examples (*mysecond.cc*)

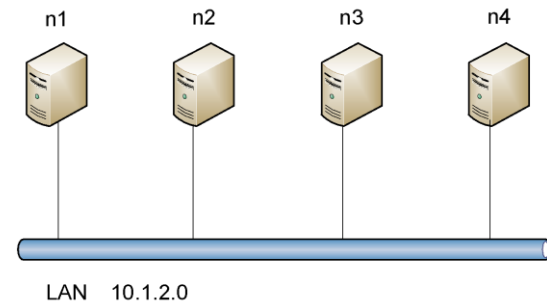


Review: CSMA topology

- CSMA network topology examples (*mysecond.cc*)
 - ✓ P2P Node creation



- ✓ Csmma Node creation



Review: CSMA Code (1/5)

```
#include "ns3/core-module.h"
#include "ns3/simulator-module.h"
#include "ns3/node-module.h"
#include "ns3/helper-module.h"

using namespace ns3;

NS_LOG_COMPONENT_DEFINE ("SecondScriptExample");

int
main (int argc, char *argv[])
{
    bool verbose = true;
    uint32_t nCsma = 3;
    CommandLine cmd;
    cmd.AddValue ("nCsma", "Number of \"extra\" CSMA nodes/devices", nCsma);
    cmd.AddValue ("verbose", "Tell echo applications to log if true", verbose);
    cmd.Parse (argc,argv);
```

Review: CSMA Code (2/5)

```
if (verbose)
{
    LogComponentEnable("UdpEchoClientApplication", LOG_LEVEL_INFO);
    LogComponentEnable("UdpEchoServerApplication", LOG_LEVEL_INFO);
}
```

```
nCsmma = nCsmma == 0 ? 1 : nCsmma;
```

```
NodeContainer p2pNodes;
```

```
p2pNodes.Create (2);
```

```
NodeContainer csmaNodes;
```

```
csmaNodes.Add (p2pNodes.Get (1));
```

```
csmaNodes.Create (nCsmma);
```

```
PointToPointHelper pointToPoint;
```

```
pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
```

```
pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));
```

Review: CSMA Code (3/5)

```
NetDeviceContainer p2pDevices;  
p2pDevices = pointToPoint.Install (p2pNodes);
```

```
CsmaHelper csma;  
csma.SetChannelAttribute ("DataRate", StringValue ("100Mbps"));  
csma.SetChannelAttribute ("Delay", TimeValue (NanoSeconds (6560)));
```

```
NetDeviceContainer csmaDevices;  
csmaDevices = csma.Install (csmaNodes);
```

```
InternetStackHelper stack;  
stack.Install (p2pNodes.Get (0));  
stack.Install (csmaNodes);
```

```
Ipv4AddressHelper address;  
address.SetBase ("10.1.1.0", "255.255.255.0");  
Ipv4InterfaceContainer p2pInterfaces;  
p2pInterfaces = address.Assign (p2pDevices);
```

Review: CSMA Code (4/5)

```
address.SetBase ("10.1.2.0", "255.255.255.0");  
Ipv4InterfaceContainer csmalInterfaces;  
csmalInterfaces = address.Assign (csmadevices);
```

```
UdpEchoServerHelper echoServer (9);
```

```
ApplicationContainer serverApps = echoServer.Install (csmadevices.Get (nCsma));  
serverApps.Start (Seconds (1.0));  
serverApps.Stop (Seconds (10.0));
```

```
UdpEchoClientHelper echoClient (csmalInterfaces.GetAddress (nCsma), 9);  
echoClient.SetAttribute ("MaxPackets", UIntegerValue (1));  
echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.)));  
echoClient.SetAttribute ("PacketSize", UIntegerValue (1024));
```

```
ApplicationContainer clientApps = echoClient.Install (p2pNodes.Get (0));  
clientApps.Start (Seconds (2.0));  
clientApps.Stop (Seconds (10.0));
```

Review: CSMA Code (5/5)

```
Ipv4GlobalRoutingHelper::PopulateRoutingTables ();
```

```
pointToPoint.EnablePcapAll ("second");
```

```
csma.EnablePcap ("second", csmaDevices.Get (1), true);
```

```
Simulator::Run ();
```

```
Simulator::Destroy ();
```

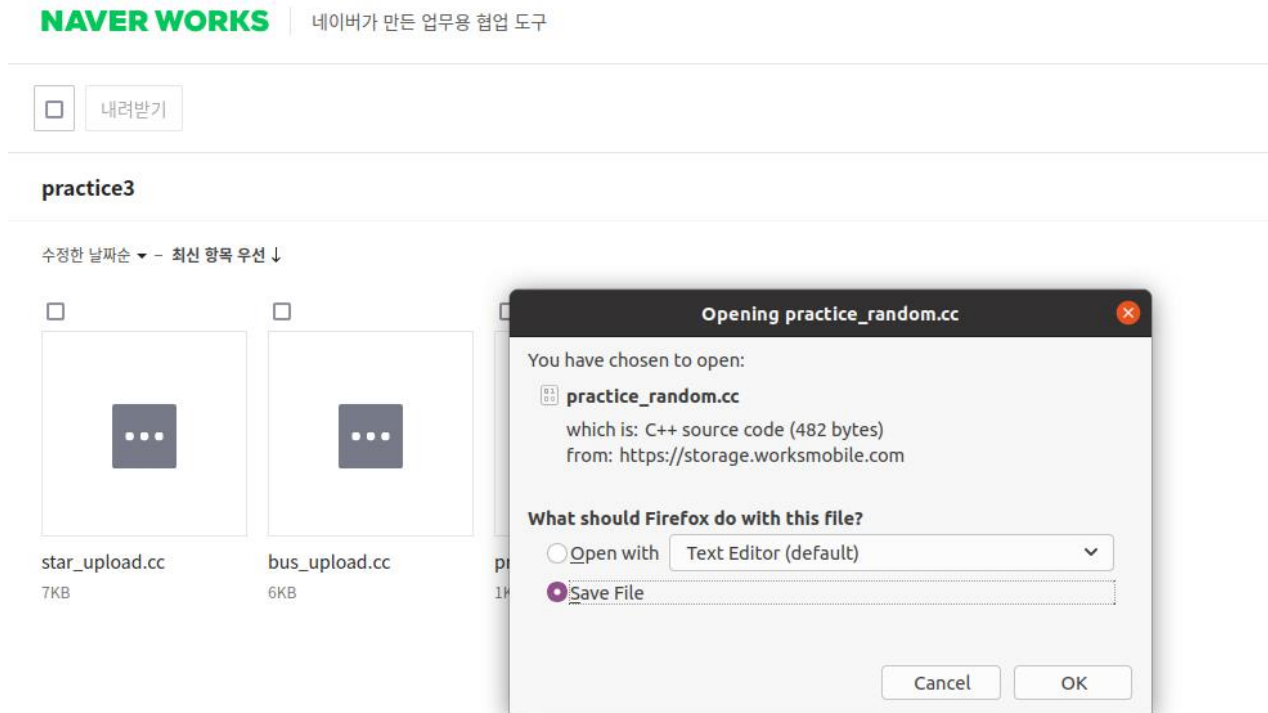
```
return 0;
```

```
}
```

Practice 3

Download .cc files (1/2)

- Download files from Naver Works Drive at VM
 - <https://works.do/5Fo1Aiw>
 - Password: 2023



Download .cc files (2/2)

- Copy and paste each file to scratch directory  Scratch 디렉토리의 path 확인 후 명령어 입력
 - ~\$ cp Downloads/practice_random.cc ns-3.35/scratch/practice_random.cc
 - ~\$ cp Downloads/practice_random.cc ns-allinone-3.35/ns-3.35/scratch/practice_random.cc

```
comnet2023@ubuntu:~$ cp Downloads/practice_random.cc ns-3.35/scratch/practice_random.cc
```

```
comnet2023@ubuntu:~/ns-3.35$ ./waf --run scratch/mysecond
Waf: Entering directory `/home/comnet2023/ns-3.35/build'
[2159/2219] Linking build/scratch/scratch-simulator
[2161/2219] Linking build/scratch/mysecond4
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/9/../../../../x86_64-linux-gnu/Scrt1.o:
in function `_start':
(.text+0x24): undefined reference to `main'
collect2: error: ld returned 1 exit status

Waf: Leaving directory `/home/comnet2023/ns-3.35/build'
Build failed
-> task in 'scratch-simulator' failed with exit status 1 (run with -v to display more information)
```

스크래치 디렉토리에 오류가 있는 파일이 있는 경우 build 오류가 발생함
코드 작성이 완료된 파일만 scratch 폴더에 옮기고 실행하기

Scenario selection

- `practice_random.cc` 파일의 `seed`에 자신의 학번 입력 후 실행

```
1
2 #include <random>
3 #include <iostream>
4
5
6
7 int main() {
8   unsigned int seed = 2024171020;
9   std::mt19937 rng(seed);
10  std::uniform_int_distribution<int> delay(1,5);
11  std::uniform_int_distribution<int> throughput(1,3);
12
13
14  int delay_option = delay(rng);
15  int throughput_option = throughput(rng);
16
17
18
19  std::cout<<"Student number : "<<seed<<std::endl;
20  std::cout<<"Delay_option : "<<delay_option<<std::endl;
21  std::cout<<"Throughput_option : "<<throughput_option<<std::endl;
22
23
24
25
26  return 0;
27 }
28
```

▲ `practice_random.cc`

자신의 학번 입력



```
comnet2023@ubuntu:~/ns-3.35$ ./waf --run scratch/practice_random
Waf: Entering directory `/home/comnet2023/ns-3.35/build'
[2156/2205] Compiling scratch/practice_random.cc
[2165/2205] Linking build/scratch/practice_random
Waf: Leaving directory `/home/comnet2023/ns-3.35/build'
Build commands will be stored in build/compile_commands.json
'build' finished successfully (1.553s)
Student number : 2024171020
Delay_option : 5
Throughput_option : 2
```

Delay_option : 1 - 5 사이의 랜덤값
Throughput_option : 1 - 3 사이의 랜덤값

▲ 출력 결과

학번에 따른 랜덤 시나리오 옵션이 출력되면, 출력 결과에 따라 실습 진행

출력 결과는 캡처하여 반드시 보고서 첫 부분에 첨부하기

Delay test scenario

- Scenario selection에서 출력된 Delay_option에 해당하는 시나리오에 맞게 코드를 작성하시기 바랍니다.

Delay_option : 1

echoClient : n1, echoServer : n2

Delay_option: 2

echoClient : n1, echoServer : n3

Delay_option : 3

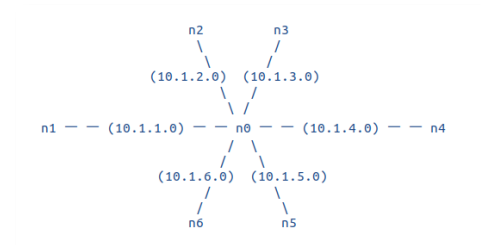
echoClient : n6, echoServer : n1

Delay_option : 4

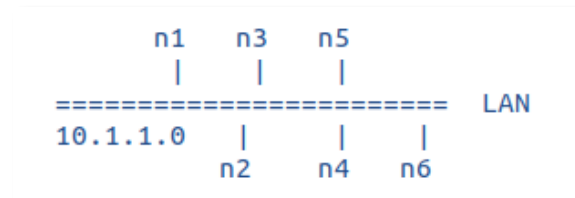
echoClient : n6, echoServer : n2

Delay_option : 5

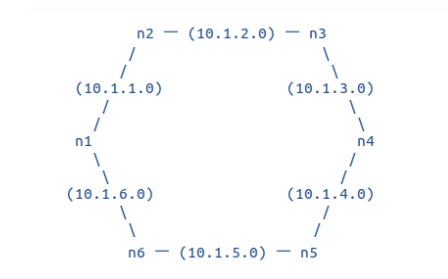
echoClient : n6, echoServer : n3



▲ star topology



▲ bus topology



▲ ring topology

Throughput test scenario

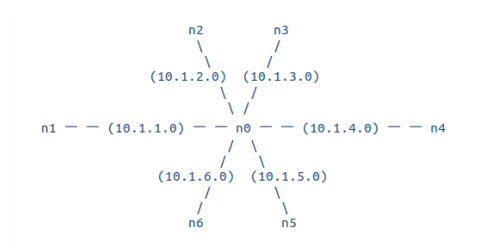
- Scenario selection에서 출력된 Throughput_option에 해당하는 시나리오에 맞게 코드를 작성하시기 바랍니다.

Throughput_option: 1

echoClient1 : n1 , echoServer1 : n4

echoClient2 : n2 , echoServer2 : n5

echoClient3 : n6 , echoServer3 : n3



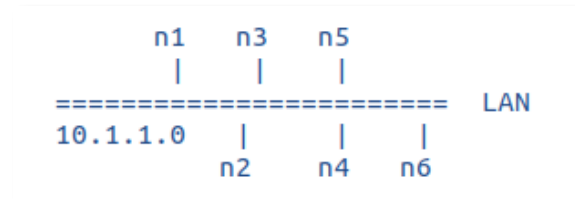
▲ star topology

Throughput_option : 2

echoClient1 : n1 , echoServer1 : n4

echoClient2 : n5 , echoServer2 : n2

echoClient3 : n3 , echoServer3 : n6



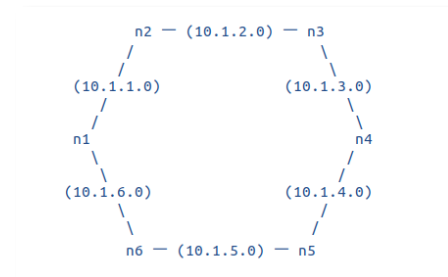
▲ bus topology

Throughput_option : 3

echoClient1 : n4 , echoServer1 : n1

echoClient2 : n2 , echoServer2 : n5

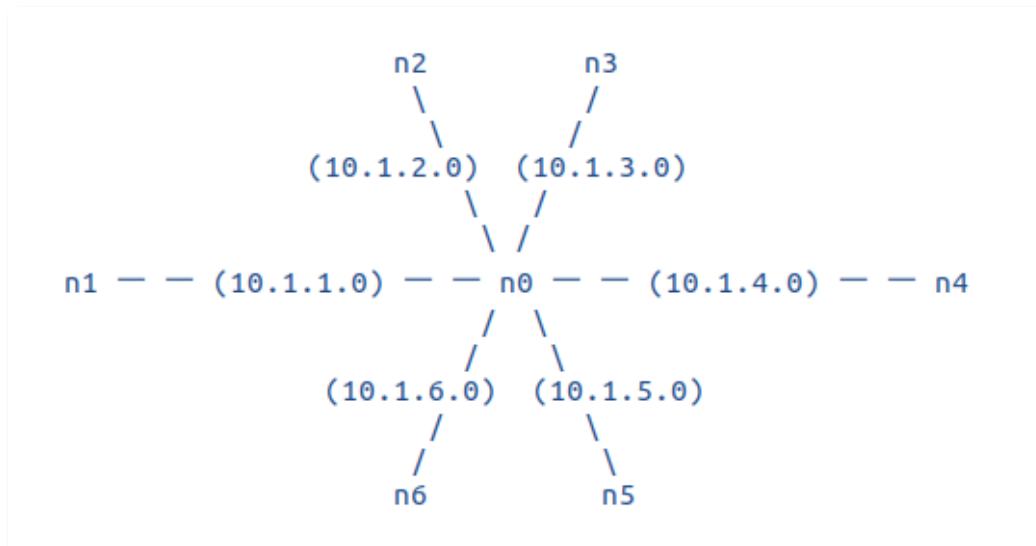
echoClient3 : n3 , echoServer3 : n6



▲ ring topology

Task 1: Complete the star topology code

- Paste the **star_upload.cc** file into the destination directory (*ns-3.35/scratch* or *ns-allinone-3.35/ns-3.35/scratch*) used in the previous practice
- Fill in the code where **// Topology Creation //** is marked



▲ star topology

Task 1: Complete the star topology code

star_upload.cc

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/csma-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/ipv4-global-routing-helper.h"
#include "ns3/netanim-module.h"
#include "ns3/mobility-module.h"
#include "ns3/flow-monitor-module.h"
```

```
using namespace ns3;
```

```
NS_LOG_COMPONENT_DEFINE ("SecondScriptExample");
```

```
int
main (int argc, char *argv[])
{
    bool verbose = true;
    float clientapptime = 2.0;
    float simTime = 11;
```

```
//Change throughputtest variable from false to true during throughput test
```

```
bool throughputtest = false;
```

Task 1: Complete the star topology code

```
CommandLine cmd;  
cmd.AddValue ("verbose", "Tell echo applications to log if true", verbose);
```

```
cmd.Parse (argc,argv);
```

```
if (verbose)  
{  
    LogComponentEnable ("UdpEchoClientApplication", LOG_LEVEL_INFO);  
    LogComponentEnable ("UdpEchoServerApplication", LOG_LEVEL_INFO);  
}
```

```
//////////////////////////////////// Topology Creation //////////////////////////////////////
```

```
** Fill in the code
```

```
// 1. Create NodeContainer (Use node 0 as a switch node)
```

```
// 2. Create P2P links and set attribution
```

```
if(!throughputtest){  
    pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("100Mbps"));  
}  
if(throughputtest){  
    pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("15kbps"));  
}  
pointToPoint.SetChannelAttribute ("Delay", StringValue ("2s"));
```

```
// 3. Create NetdeviceContainer
```

```
// 4. Create InternetStack and assign IP addresses
```


Task 1: Complete the star topology code

// 5-1. Application for delay test

```
if (!throughputtest){
```

```
// Packet 1
```

```
UdpEchoServerHelper echoServer1 (9);
```

```
ApplicationContainer serverApps1 = echoServer1.Install (p2pNodes.Get (4));
```

```
serverApps1.Start (Seconds (1.0));
```

```
serverApps1.Stop (Seconds (simTime));
```

```
UdpEchoClientHelper echoClient1 (p2pInterfaces04.GetAddress (1), 9);
```

```
echoClient1.SetAttribute ("MaxPackets", UIntegerValue (1));
```

```
echoClient1.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
```

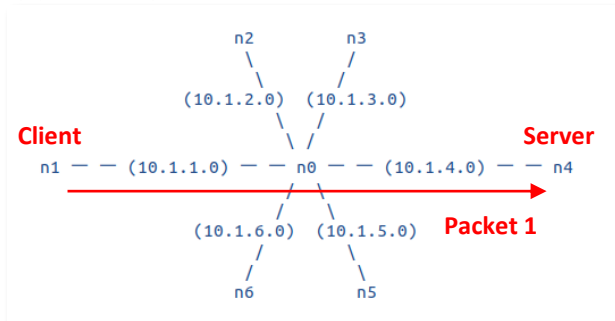
```
echoClient1.SetAttribute ("PacketSize", UIntegerValue (1024));
```

```
ApplicationContainer clientApps1 = echoClient1.Install (p2pNodes.Get (1));
```

```
clientApps1.Start (Seconds (clientapptime));
```

```
clientApps1.Stop (Seconds (simTime));
```

```
}
```



▲ Delay test 예시

회색 코드는 예시에 대한 코드를 작성한 부분입니다.

학생들은 각자의 option 값에 맞게 회색 부분을

작성해주세요.

Task 1: Complete the star topology code

// 5-2. Application for throughput test

```
if (throughputtest){
```

```
// Packet 1
```

```
UdpEchoServerHelper echoServer1 (9);
```

```
ApplicationContainer serverApps1 = echoServer1.Install (p2pNodes.Get (4));
```

```
serverApps1.Start (Seconds (1.0));
```

```
serverApps1.Stop (Seconds (simTime));
```

```
UdpEchoClientHelper echoClient1 (p2pInterfaces04.GetAddress (1), 9);
```

```
echoClient1.SetAttribute ("MaxPackets", UIntegerValue (3));
```

```
echoClient1.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
```

```
echoClient1.SetAttribute ("PacketSize", UIntegerValue (1024));
```

```
ApplicationContainer clientApps1 = echoClient1.Install (p2pNodes.Get (1));
```

```
clientApps1.Start (Seconds (clientapptime));
```

```
clientApps1.Stop (Seconds (simTime));
```

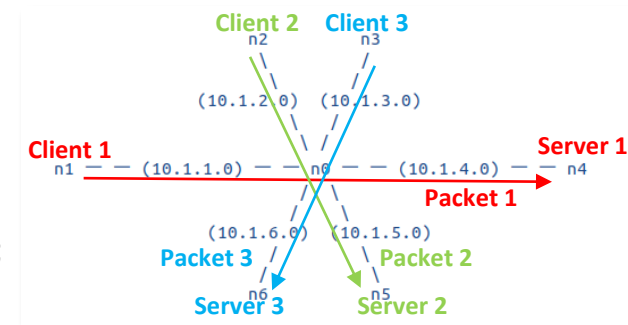
```
// Packet 2
```

```
UdpEchoServerHelper echoServer2 (10);
```

```
ApplicationContainer serverApps2 = echoServer2.Install (p2pNodes.Get (5));
```

```
serverApps2.Start (Seconds (1.0));
```

```
serverApps2.Stop (Seconds (simTime));
```



▲ Throughput test 예시

Task 1: Complete the star topology code

```
UdpEchoClientHelper echoClient2 (p2pInterfaces05.GetAddress (1), 10);
echoClient2.SetAttribute ("MaxPackets", UIntegerValue (3));
echoClient2.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
echoClient2.SetAttribute ("PacketSize", UIntegerValue (1024));
```

```
ApplicationContainer clientApps2 = echoClient2.Install (p2pNodes.Get (2));
clientApps2.Start (Seconds (clientapptime));
clientApps2.Stop (Seconds (simTime));
```

// Packet 3

```
UdpEchoServerHelper echoServer3 (11);
ApplicationContainer serverApps3 = echoServer3.Install (p2pNodes.Get (6));
serverApps3.Start (Seconds (1.0));
serverApps3.Stop (Seconds (simTime));
```

```
UdpEchoClientHelper echoClient3 (p2pInterfaces06.GetAddress (1), 11);
echoClient3.SetAttribute ("MaxPackets", UIntegerValue (3));
echoClient3.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
echoClient3.SetAttribute ("PacketSize", UIntegerValue (1024));
```

```
ApplicationContainer clientApps3 = echoClient3.Install (p2pNodes.Get (3));
clientApps3.Start (Seconds (clientapptime));
clientApps3.Stop (Seconds (simTime));
}
```

//////////////////////////////// End //////////////////////////////////

Task 1: Complete the star topology code

```
Ipv4GlobalRoutingHelper::PopulateRoutingTables ();
```

```
FlowMonitorHelper flowmonHelper;  
Ptr<FlowMonitor> monitor = flowmonHelper.InstallAll();
```

```
Simulator::Stop(Seconds(100.0));  
Simulator::Run ();  
pointToPoint.EnablePcapAll ("second");
```

```
// Flowmonitor for throughput and delay output
```

```
monitor->CheckForLostPackets();  
Ptr<Ipv4FlowClassifier> classifier =  
DynamicCast<Ipv4FlowClassifier>(flowmonHelper.GetClassifier());  
FlowMonitor::FlowStatsContainer stats = monitor->GetFlowStats();
```

Task 1: Complete the star topology code

```
for (std::map<FlowId, FlowMonitor::FlowStats>::const_iterator iter = stats.begin(); iter != stats.end(); ++iter) {
    Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow (iter->first);

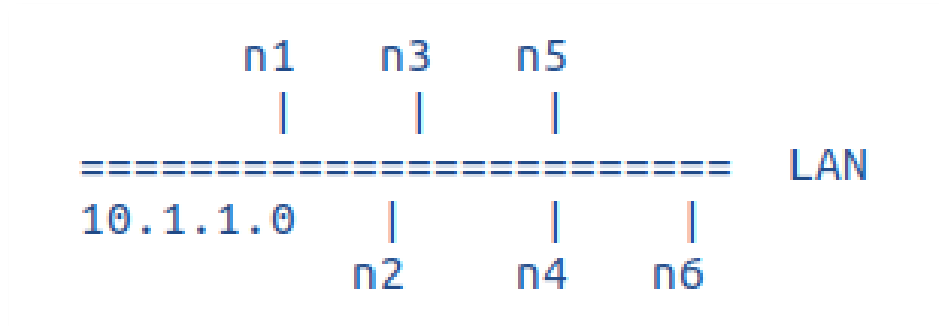
    std::cout << "Flow " << t.sourceAddress << " -> " << t.destinationAddress << "\n";
    std::cout << " Tx Bytes: " << iter->second.txBytes << "\n";
    std::cout << " Throughput: " << iter->second.rxBytes * 8.0 / rxDuration / 1000.0 << "Kbps\n";
    std::cout << " Mean delay: " << 1000 * iter->second.delaySum.GetSeconds () / iter->second.rxPackets << " ms\n";

}

Simulator::Destroy ();
return 0;
}
```

Task 2: Complete the bus topology code

- Paste the **bus_upload.cc** file into the destination directory (*ns-3.35/scratch* or *ns-allinone-3.35/ns-3.35/scratch*) used in the previous practice
- Fill in the code where **// Create Topology //** is marked



▲ bus topology

Task 2: Complete the bus topology code

bus_upload.cc

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/csma-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"
#include "ns3/ipv4-global-routing-helper.h"
#include "ns3/netanim-module.h"
#include "ns3/mobility-module.h"
#include "ns3/flow-monitor-module.h"
```

```
using namespace ns3;
```

```
NS_LOG_COMPONENT_DEFINE ("SecondScriptExample");
```

```
int
main (int argc, char *argv[])
{
    bool verbose = true;
    float clientapptime = 2.0;
    double delay = 0.004;
    uint32_t nCsma = 6;
```

```
//Change throughputtest variable from false to true during
```

```
bool throughputtest = false;
```

Task 2: Complete the bus topology code

```
CommandLine cmd;  
cmd.AddValue ("verbose", "Tell echo applications to log if true", verbose);  
  
cmd.Parse (argc,argv);  
  
if (verbose)  
{  
    LogComponentEnable ("UdpEchoClientApplication", LOG_LEVEL_INFO);  
    LogComponentEnable ("UdpEchoServerApplication", LOG_LEVEL_INFO);  
}  
  
nCsma = nCsma == 0 ? 1 : nCsma;
```

//////////////////////////////////// Topology Creation //////////////////////////////////////

// 1. Create NodeContainer

// 2. Create CSMA link and **set attribution**

```
if(!throughputtest){  
    csma.SetChannelAttribute ("DataRate", StringValue ("100Mbps"))  
}  
if(throughputtest){  
    csma.SetChannelAttribute ("DataRate", StringValue ("15kbps"));  
}
```

3. Create NetDeviceContainer

4. Create InternetStack and assign IP addresses

Task 2: Complete the bus topology code

// 5-1. Application for delay test

```
if(!throughputtest){
```

```
// Packet 1
```

```
UdpEchoServerHelper echoServer1 (9);
```

```
ApplicationContainer serverApps1 = echoServer1.Install (csmaNodes.Get (3));
```

```
serverApps1.Start (Seconds (1.0));
```

```
serverApps1.Stop (Seconds (simTime));
```

```
UdpEchoClientHelper echoClient1 (csmaInterfaces.GetAddress (3), 9);
```

```
echoClient1.SetAttribute ("MaxPackets", UIntegerValue (1));
```

```
echoClient1.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
```

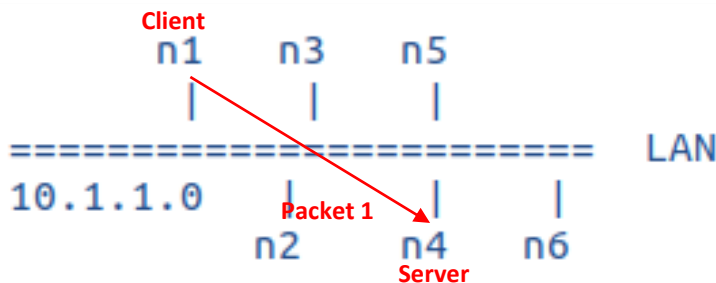
```
echoClient1.SetAttribute ("PacketSize", UIntegerValue (1024));
```

```
ApplicationContainer clientApps1 = echoClient1.Install (csmaNodes.Get (0));
```

```
clientApps1.Start (Seconds (clientapptime));
```

```
clientApps1.Stop (Seconds (simTime));
```

```
}
```



회색 코드는 예시에 대한 코드를 작성한 부분입니다.

학생들은 각자의 option 값에 맞게 회색 부분을

작성해주세요.

Task 2: Complete the bus topology code

// 5-2 Application for throughput test

```
if(throughputtest){
```

// Packet 1

```
UdpEchoServerHelper echoServer1 (9);
```

```
ApplicationContainer serverApps1 = echoServer1.Install (csmaNodes.Get (3));
```

```
serverApps1.Start (Seconds (1.0));
```

```
serverApps1.Stop (Seconds (simTime));
```

```
UdpEchoClientHelper echoClient1 (csmaInterfaces.GetAddress (3), 9);
```

```
echoClient1.SetAttribute ("MaxPackets", UIntegerValue (3));
```

```
echoClient1.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
```

```
echoClient1.SetAttribute ("PacketSize", UIntegerValue (1024));
```

```
ApplicationContainer clientApps1 = echoClient1.Install (csmaNodes.Get (0));
```

```
clientApps1.Start (Seconds (clientapptime));
```

```
clientApps1.Stop (Seconds (simTime));
```

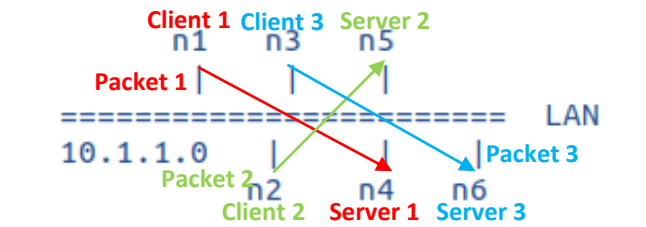
// Packet 2

```
UdpEchoServerHelper echoServer2 (10);
```

```
ApplicationContainer serverApps2 = echoServer2.Install (csmaNodes.Get (4));
```

```
serverApps2.Start (Seconds (1.0));
```

```
serverApps2.Stop (Seconds (simTime));
```



▲ Throughput test 예시

Task 2: Complete the bus topology code

```
UdpEchoClientHelper echoClient2 (csmaInterfaces.GetAddress (4), 10);
echoClient2.SetAttribute ("MaxPackets", UIntegerValue (3));
echoClient2.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
echoClient2.SetAttribute ("PacketSize", UIntegerValue (1024));
```

```
ApplicationContainer clientApps2 = echoClient2.Install (csmaNodes.Get (1));
clientApps2.Start (Seconds (clientapptime));
clientApps2.Stop (Seconds (simTime));
```

// Packet 3

```
UdpEchoServerHelper echoServer3 (11);
ApplicationContainer serverApps3 = echoServer3.Install (csmaNodes.Get (5));
serverApps3.Start (Seconds (1.0));
serverApps3.Stop (Seconds (simTime));
```

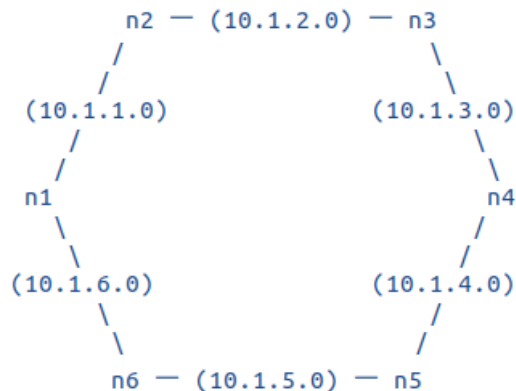
```
UdpEchoClientHelper echoClient3 (csmaInterfaces.GetAddress (5), 11);
echoClient3.SetAttribute ("MaxPackets", UIntegerValue (3));
echoClient3.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
echoClient3.SetAttribute ("PacketSize", UIntegerValue (1024));
```

```
ApplicationContainer clientApps3 = echoClient3.Install (csmaNodes.Get (2));
clientApps3.Start (Seconds (clientapptime));
clientApps3.Stop (Seconds (simTime));
}
```

//////////////////////////////////// End //////////////////////////////////////

Task 3: Complete the ring topology code

- Create **ring_upload.cc** in scratch directory (*ns-3.35/scratch* or *ns-allinone-3.35/ns-3.35/scratch*)
- **Write your own code** referring to task 1 and task 2



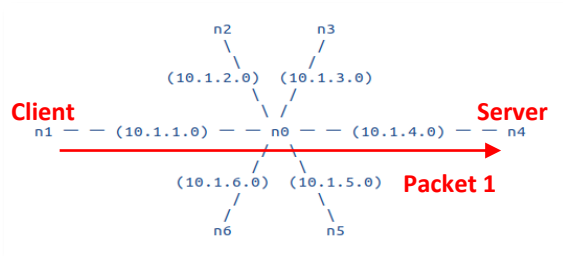
▲ ring topology

```
comnet2023@ubuntu:~/ns-3.35$ cd scratch  
comnet2023@ubuntu:~/ns-3.35/scratch$ gedit ring_upload.cc
```

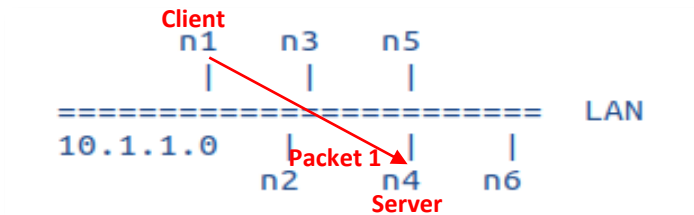
You can use gedit command to create new file

Task 4: Delay test

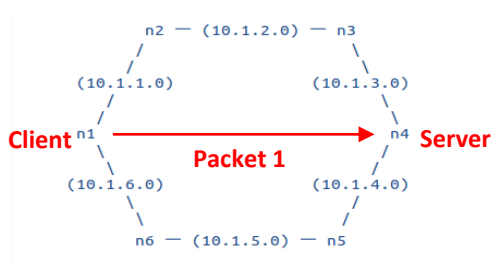
- 코드에서 **throughputtest = false**인지 확인 후 각 토폴로지를 실행하세요.
- 자신의 Delay_option에 맞게 작성한 코드를 실행하고, 해당 결과에 대한 이유를 분석하여 적어주세요.
- 다음은 packet 1을 n1에서 n4로 보내는 코드에 대한 결과 예시입니다.



▲ star topology



▲ bus topology



▲ ring topology

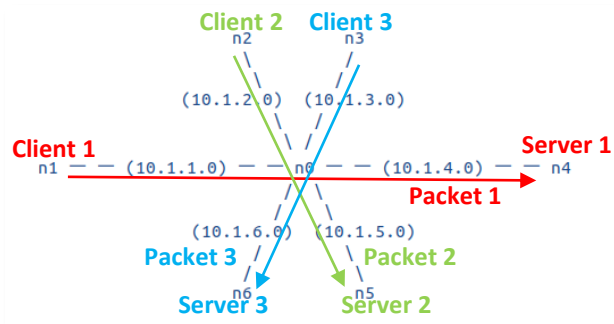
```
comnet2023@ubuntu:~/ns-3.35$ ./waf --run scratch/star_upload
Waf: Entering directory '/home/comnet2023/ns-3.35/build'
[2157/2211] Compiling scratch/star_upload.cc
Waf: Leaving directory '/home/comnet2023/ns-3.35/build'
Build commands will be stored in build/compile_commands.json
'build' finished successfully (2.894s)
At time +2s client sent 1024 bytes to 10.1.4.2 port 9
At time +6.00017s server received 1024 bytes from 10.1.1.2 port 49153
At time +6.00017s server sent 1024 bytes to 10.1.1.2 port 49153
At time +10.0003s client received 1024 bytes from 10.1.4.2 port 9
Flow 10.1.1.2 -> 10.1.4.2
Tx Bytes: 1052
Throughput: 0.93511Kbps
Mean delay: 4000.17 ms
Flow 10.1.4.2 -> 10.1.1.2
Tx Bytes: 1052
Throughput: 0.93511Kbps
Mean delay: 4000.17 ms
```

```
comnet2023@ubuntu:~/ns-3.35$ ./waf --run scratch/bus_upload
Waf: Entering directory '/home/comnet2023/ns-3.35/build'
Waf: Leaving directory '/home/comnet2023/ns-3.35/build'
Build commands will be stored in build/compile_commands.json
'build' finished successfully (0.291s)
At time +2s client sent 1024 bytes to 10.1.1.4 port 9
At time +2.0071s server received 1024 bytes from 10.1.1.1 port 49153
At time +2.0071s server sent 1024 bytes to 10.1.1.1 port 49153
At time +2.0112s client received 1024 bytes from 10.1.1.4 port 9
Flow 10.1.1.1 -> 10.1.1.4
Tx Bytes: 1052
Throughput: 0.93511Kbps
Mean delay: 7.09784 ms
Flow 10.1.1.4 -> 10.1.1.1
Tx Bytes: 1052
Throughput: 0.93511Kbps
Mean delay: 4.09784 ms
```

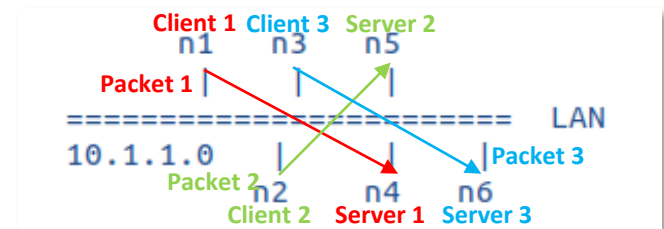
```
comnet2023@ubuntu:~/ns-3.35$ ./waf --run scratch/ring_upload
Waf: Entering directory '/home/comnet2023/ns-3.35/build'
Waf: Leaving directory '/home/comnet2023/ns-3.35/build'
Build commands will be stored in build/compile_commands.json
'build' finished successfully (0.299s)
At time +2s client sent 1024 bytes to 10.1.4.1 port 9
At time +8.00025s server received 1024 bytes from 10.1.1.1 port 49153
At time +8.00025s server sent 1024 bytes to 10.1.1.1 port 49153
At time +14.0005s client received 1024 bytes from 10.1.3.2 port 9
Flow 10.1.1.1 -> 10.1.4.1
Tx Bytes: 1052
Throughput: 0.93511Kbps
Mean delay: 6000.25 ms
Flow 10.1.3.2 -> 10.1.1.1
Tx Bytes: 1052
Throughput: 0.93511Kbps
Mean delay: 6000.25 ms
```

Task 5: Throughput test

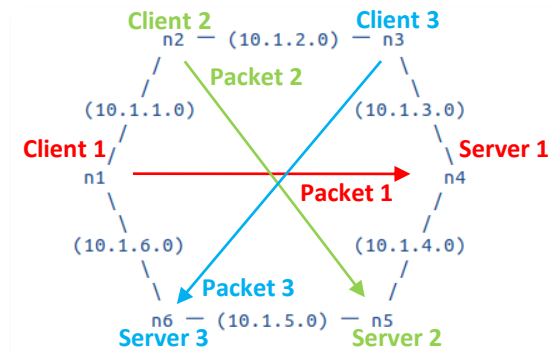
- 코드에서 **throughputtest = true**로 변경 후 각 토폴로지를 실행하세요.
- 자신의 Throughput_option에 맞게 작성한 코드를 실행하고, 해당 결과에 대한 이유를 분석하여 적어주세요.
- 다음은 3개의 packet을 보내는 예시입니다.



▲ star topology



▲ bus topology



▲ ring topology

Task 5: Throughput test

```
At time +2s client sent 1024 bytes to 10.1.4.2 port 9
At time +2s client sent 1024 bytes to 10.1.5.2 port 10
At time +2s client sent 1024 bytes to 10.1.6.2 port 11
At time +3s client sent 1024 bytes to 10.1.4.2 port 9
At time +3s client sent 1024 bytes to 10.1.5.2 port 10
At time +3s client sent 1024 bytes to 10.1.6.2 port 11
At time +4s client sent 1024 bytes to 10.1.4.2 port 9
At time +4s client sent 1024 bytes to 10.1.5.2 port 10
At time +4s client sent 1024 bytes to 10.1.6.2 port 11
At time +7.12427s server received 1024 bytes from 10.1.1.2 port 49153
At time +7.12427s server sent 1024 bytes to 10.1.1.2 port 49153
At time +7.12427s server received 1024 bytes from 10.1.2.2 port 49153
At time +7.12427s server sent 1024 bytes to 10.1.2.2 port 49153
At time +7.12427s server received 1024 bytes from 10.1.3.2 port 49153
At time +7.12427s server sent 1024 bytes to 10.1.3.2 port 49153
At time +8.12427s server received 1024 bytes from 10.1.1.2 port 49153
At time +8.12427s server sent 1024 bytes to 10.1.1.2 port 49153
At time +8.12427s server received 1024 bytes from 10.1.2.2 port 49153
At time +8.12427s server sent 1024 bytes to 10.1.2.2 port 49153
At time +8.12427s server received 1024 bytes from 10.1.3.2 port 49153
At time +8.12427s server sent 1024 bytes to 10.1.3.2 port 49153
At time +9.12427s server received 1024 bytes from 10.1.1.2 port 49153
At time +9.12427s server sent 1024 bytes to 10.1.1.2 port 49153
At time +9.12427s server received 1024 bytes from 10.1.2.2 port 49153
At time +9.12427s server sent 1024 bytes to 10.1.2.2 port 49153
At time +9.12427s server received 1024 bytes from 10.1.3.2 port 49153
At time +9.12427s server sent 1024 bytes to 10.1.3.2 port 49153
Flow 10.1.1.2 -> 10.1.4.2
Tx Bytes: 3156
Throughput: 2.80533Kbps
Mean delay: 5124.27 ms
Flow 10.1.2.2 -> 10.1.5.2
Tx Bytes: 3156
Throughput: 2.80533Kbps
Mean delay: 5124.27 ms
Flow 10.1.3.2 -> 10.1.6.2
Tx Bytes: 3156
Throughput: 2.80533Kbps
Mean delay: 5124.27 ms
Flow 10.1.4.2 -> 10.1.1.2
Tx Bytes: 3156
Throughput: 2.80533Kbps
Mean delay: 5124.27 ms
Flow 10.1.5.2 -> 10.1.1.2
Tx Bytes: 3156
Throughput: 2.80533Kbps
Mean delay: 5124.27 ms
Flow 10.1.6.2 -> 10.1.1.2
Tx Bytes: 3156
Throughput: 2.80533Kbps
Mean delay: 5124.27 ms
```

▲ star topology

```
At time +2s client sent 1024 bytes to 10.1.1.4 port 9
At time +2s client sent 1024 bytes to 10.1.1.5 port 10
At time +2s client sent 1024 bytes to 10.1.1.6 port 11
At time +2.63894s server received 1024 bytes from 10.1.1.2 port 49153
At time +2.63894s server sent 1024 bytes to 10.1.1.2 port 49153
At time +3s client sent 1024 bytes to 10.1.1.4 port 9
At time +3s client sent 1024 bytes to 10.1.1.5 port 10
At time +3s client sent 1024 bytes to 10.1.1.6 port 11
At time +3.27887s client received 1024 bytes from 10.1.1.5 port 10
At time +3.91791s server received 1024 bytes from 10.1.1.3 port 49153
At time +3.91791s server sent 1024 bytes to 10.1.1.3 port 49153
At time +4s client sent 1024 bytes to 10.1.1.4 port 9
At time +4s client sent 1024 bytes to 10.1.1.5 port 10
At time +4s client sent 1024 bytes to 10.1.1.6 port 11
At time +4.52815s server received 1024 bytes from 10.1.1.3 port 49153
At time +4.52815s server sent 1024 bytes to 10.1.1.3 port 49153
At time +5.13935s client received 1024 bytes from 10.1.1.6 port 11
At time +5.77829s server received 1024 bytes from 10.1.1.1 port 49153
At time +5.77829s server sent 1024 bytes to 10.1.1.1 port 49153
At time +6.38735s server received 1024 bytes from 10.1.1.1 port 49153
At time +6.38735s server sent 1024 bytes to 10.1.1.1 port 49153
At time +6.96442s server received 1024 bytes from 10.1.1.1 port 49153
At time +6.96442s server sent 1024 bytes to 10.1.1.1 port 49153
At time +7.60424s client received 1024 bytes from 10.1.1.4 port 9
At time +8.1813s client received 1024 bytes from 10.1.1.4 port 9
At time +8.75837s client received 1024 bytes from 10.1.1.4 port 9
Flow 10.1.1.1 -> 10.1.1.4
Tx Bytes: 3156
Throughput: 2.80533Kbps
Mean delay: 3376.69 ms
Flow 10.1.1.2 -> 10.1.1.5
Tx Bytes: 3156
Throughput: 0.935111Kbps
Mean delay: 638.935 ms
Flow 10.1.1.3 -> 10.1.1.6
Tx Bytes: 3156
Throughput: 1.87022Kbps
Mean delay: 1723.03 ms
Flow 10.1.1.5 -> 10.1.1.2
Tx Bytes: 1052
Throughput: 0.935111Kbps
Mean delay: 639.935 ms
Flow 10.1.1.6 -> 10.1.1.3
Tx Bytes: 2104
Throughput: 0.935111Kbps
Mean delay: 1221.44 ms
Flow 10.1.1.4 -> 10.1.1.1
Tx Bytes: 3156
Throughput: 2.80533Kbps
Mean delay: 1804.62 ms
```

▲ bus topology

```
At time +2s client sent 1024 bytes to 10.1.4.1 port 9
At time +2s client sent 1024 bytes to 10.1.5.1 port 10
At time +2s client sent 1024 bytes to 10.1.6.1 port 11
At time +3s client sent 1024 bytes to 10.1.4.1 port 9
At time +3s client sent 1024 bytes to 10.1.5.1 port 10
At time +3s client sent 1024 bytes to 10.1.6.1 port 11
At time +4s client sent 1024 bytes to 10.1.4.1 port 9
At time +4s client sent 1024 bytes to 10.1.5.1 port 10
At time +4s client sent 1024 bytes to 10.1.6.1 port 11
At time +9.6864s server received 1024 bytes from 10.1.1.1 port 49153
At time +9.6864s server sent 1024 bytes to 10.1.1.1 port 49153
At time +9.6864s server received 1024 bytes from 10.1.1.2 port 49153
At time +9.6864s server sent 1024 bytes to 10.1.1.2 port 49153
At time +9.6864s server received 1024 bytes from 10.1.1.2 port 49153
At time +9.6864s server sent 1024 bytes to 10.1.2.2 port 49153
At time +9.6864s server sent 1024 bytes to 10.1.2.2 port 49153
At time +10.6864s server received 1024 bytes from 10.1.1.1 port 49153
At time +10.6864s server sent 1024 bytes to 10.1.1.1 port 49153
At time +10.6864s server received 1024 bytes from 10.1.1.2 port 49153
At time +10.6864s server sent 1024 bytes to 10.1.1.2 port 49153
At time +10.6864s server received 1024 bytes from 10.1.2.2 port 49153
At time +10.6864s server sent 1024 bytes to 10.1.2.2 port 49153
Flow 10.1.1.1 -> 10.1.4.1
Tx Bytes: 3156
Throughput: 2.80533Kbps
Mean delay: 7686.4 ms
Flow 10.1.1.2 -> 10.1.5.1
Tx Bytes: 3156
Throughput: 2.80533Kbps
Mean delay: 7686.4 ms
Flow 10.1.2.2 -> 10.1.6.1
Tx Bytes: 3156
Throughput: 2.80533Kbps
Mean delay: 7686.4 ms
Flow 10.1.3.2 -> 10.1.1.1
Tx Bytes: 2104
Throughput: 1.87022Kbps
Mean delay: 7686.4 ms
Flow 10.1.4.2 -> 10.1.1.2
Tx Bytes: 2104
Throughput: 1.87022Kbps
Mean delay: 7686.4 ms
Flow 10.1.5.2 -> 10.1.2.2
Tx Bytes: 2104
Throughput: 1.87022Kbps
Mean delay: 7686.4 ms
```

▲ ring topology

Assignment (1/2)

1. Complete the **star topology code**

2. Complete the **bus topology code**

3. Complete the **ring topology code**

4. Conduct **delay tests**:

Perform tests on **star, bus, and ring topologies** and compare the results according to your **Delay_option**

5. Conduct **throughput tests**:

Perform tests on **star, bus, and ring topologies** and compare the results according to your **Throughput_option**

- **Results of practice_random** must be included on the first part of the report
- Task 1, 2, 3: **Capture modified parts and execution results**, and attach it
- Task 4, 5: **Capture the execution results** for each topology and attach it
Compare the results for each topology

Assignment (2/2)

- Files to Submit: report, code zip file
- File Format:
 1. Report – **HW3_your Student ID_your Name.pdf** (e.g. HW3_20XXXXXXXXX_김기훈.pdf)
 2. Zip File – **Topology_your Student ID_your Name.zip** (Star, Bus, Ring code file 포함)
- Submit Location: **Blackboard**
- Deadline: **12/15 11:59 PM**
- Total score: 30
- **본인이 작성한 코드가 아닐 경우, 모든 실습 (Practice 2 and 3)에 대한 점수가 0점 처리됩니다.**